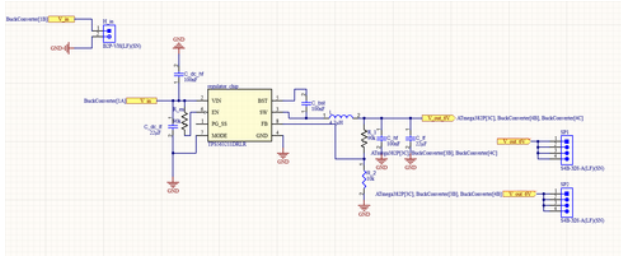
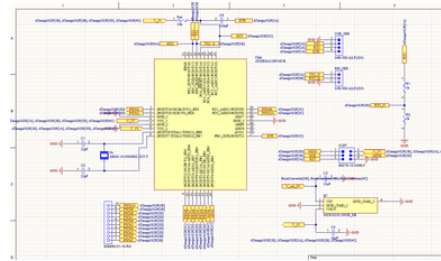


SERVO CONTROL BOARD - DATA SCIENCE AND INFORMATICS LAB

Buck Converter schematic



MCU integration schematic + LDO



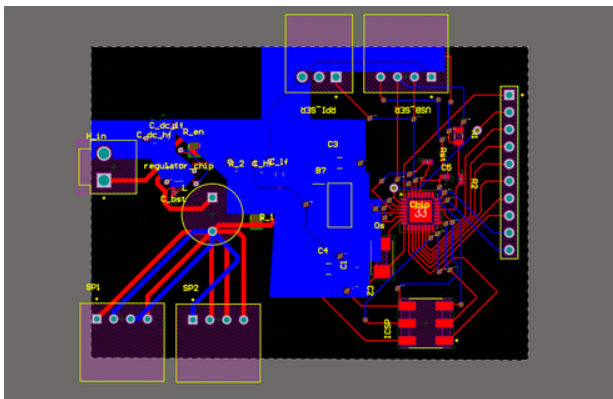
Situation

- Needed a centralized power, control, and communication system to work as a URDF robot custom end-effector
- Developed a control board with an in-built **voltage regulation, programming capabilities**, and **UART** communication

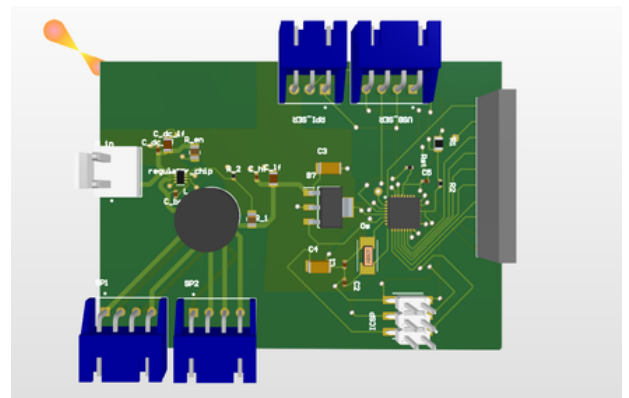
Method

- Designed all schematics in **Altium Designer**, referencing IC **datasheets** for circuit design, leading to a final **PCB design**
- Implemented **ATmega328P MCU** due to extensive community documentation and support, A **6V 3A switching buck converter** (TPS563211) to handle the peak stall currents of ~2A, **5V LDO linear voltage regulator** (MCP1815S) to safely power the ATmega

2D Altium PCB View



3D Altium PCB View



Results

- **Iterated** through multiple PCB designs until all features were functional
- Implemented **Root Cause Analysis** principles to diagnose PCB issues, systematically tracing ideal behavior and matching with actual behavior, utilizing a **Digital Multimeter**
- Developed a final working **2-layer** PCB with all features fully functional

Features

- **Voltage regulation** system utilizing a buck converter and a linear voltage regulator to safely power servos and the ATmega
- Enabled **UART** communication for MCU for firmware uploading and Rpi communication, adding a **resistive voltage** divider on TX Line, stepping down ATmega's 5V output to 3.3V, protecting the Raspberry Pi's GPIO from overvoltage allowing for safe communication between ATmega and Rpi
- **ICSP** Header for initial **bootloader** burning without the need for a pre-flashed chip

MICROMOUSE - UGA IEEE

International MicroMouse Competition



Navigation Feedback Loop Code

```
110 //reset 16 sensors with ADC_WKUP using I2C
111 digitalWrite(sha24, DATA_ADC_WKUP, LOW);
112 //initialize internal representation grids
113 #ifndef FLOODFILL
114 //the representations are the maze itself, and the floodfill version of the maze
115 rep_flood = #floodfill;
116 rep_walls = maze_grid;
117 #else
118 //initialize floodfill grid (rep);
119 #endif
120 while (1)
121 {
122     //get information from all 16 sensors
123     for (int i = 0; i < 16; i++)
124     {
125         //if sensors are pre-set configured to follow RL_028 config
126         RL_028 dir = 1; //if sensors are pre-set configured to follow RL_028 config
127
128         //wall exists in direction
129         if (digitalRead(RELEASED_IR_ADC_WKUP))
130         {
131             //from direction of sensor's direction
132             ABS_dir = solve_direction(direction: dir, clamped_heading: heading);
133
134             //update internal representations
135             updateMaze(wall_location: true_dir, maze_grid: rep_walls, x: rep.x, y: rep.y);
136             updateLocation(direction: true_dir, grid: rep_flood, heading: x: rep.x, y: rep.y);
137             //walls are defaulted to not exist, so no further code is necessary
138         }
139     }
140
141     //find where the bot should move relatively
142     RL_028 path = findPath(grid: rep_flood);
143
144     //travel in said direction
145     travelPath(dir: path, gpio_gpio: MOTOR_PINS);
146
147     //update location accordingly
148     updateLocation(dir: path, x: rep.x, y: rep.y);
149     //delay (delay: 500);
150 }
```

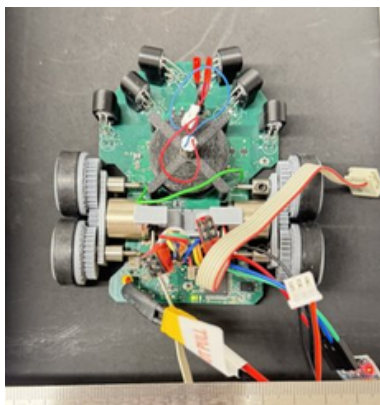
Method

- Utilized the **STM32 MCU** as the target hardware to develop all firmware
- Managed concurrent **I2C**, **UART PWM**, and **ADC peripherals** on STM32 using CubeMX and **HAL** drivers for IMU heading, Bluetooth telemetry, motor control, and wall detection
- Integrated all features into a **floodfill navigation firmware** in **embedded C**

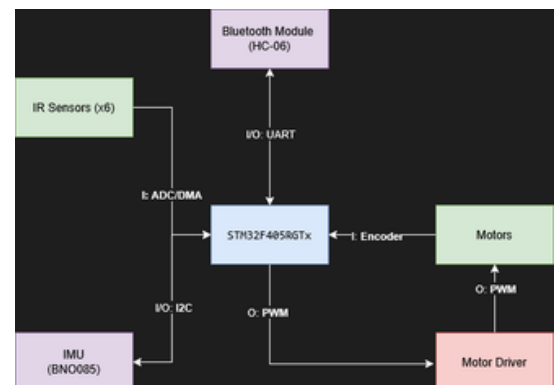
Situation

- Needed autonomous maze-solving firmware for a 16x16 competition maze with **real-time** wall detection and **dynamic path calculation**
- Required a navigation firmware that could run efficiently within **limited RAM** without blocking time-critical **peripheral communication**

Fully developed MicroMouse robot



Sensor Communication Diagram



Results

- Demonstrated **full maze solve** on test runs before competition, with firmware reliably managing all peripherals simultaneously under real conditions
- Implemented **live Bluetooth telemetry** to validate firmware state during development, confirming correct peripheral timing and navigation decisions in real time

Features

- Multi-peripheral firmware architecture** coordinating ADC, I2C, UART, and PWM simultaneously on a single STM32
- Real-time floodfill execution** within RAM and CPU constraints leveraging **DMA**-offloaded sensor reads, allowing for non-blocking simultaneous peripheral communication
- Wireless debug pipeline** through UART/Bluetooth for **live firmware state monitoring** during testing